LANGCHAIN ACADEMY · SESSION 12

Production Monitoring for AI Agents

Track what an agent costs and how well it works once real users arrive, using LangSmith.

Module 1 · Cost + Module 3 · Quality

THE COURSE

Where today fits

Module 1

Cost

- Token and cost breakdowns
- Tracking costs
- Viewing costs

Today

Module 2

Product analytics

- Analyze trends with Insights

Not today

Module 3

Quality

- User sentiment
- Errors and latency
- A/B testing

Today

Module 4

Security

- Monitoring security
- Setting up alerts

Not today

Module 1

Monitoring cost

Token and cost breakdowns · Tracking costs · Viewing costs

WHY IT MATTERS

Agent costs grow with usage

RECORDED FOR YOU

Model calls

Token usage and cost are captured automatically for major providers

+

SENT BY YOU

Everything else

Custom cost data for tools, retrieval and other components

=

IN LANGSMITH

One view of spend

Across the whole application, to monitor, understand and debug

Usage-based costs are easy to lose track of — so put them all in one place.

LESSON · TOKEN AND COST BREAKDOWNS

Three cost categories

Input

Tokens in the prompt sent to the model

cache reads · text · image

Output

Tokens the model generates in its response

reasoning · text · image

Other

Costs that aren't model tokens

tool calls · retrieval · custom runs

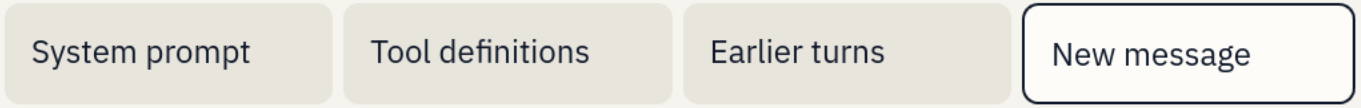
EXAMPLE FROM THE DOCS · HOVER A SECTION IN THE UI TO SEE THIS



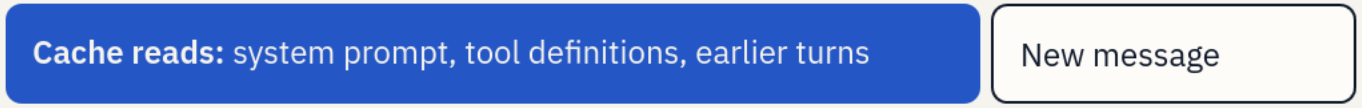
LESSON · TOKEN AND COST BREAKDOWNS

What are cache reads?

CALL 1 · EVERY TOKEN PROCESSED



CALL 2 · SAME PREFIX COMES FROM THE CACHE



50–90%

cheaper than regular input tokens, depending on the provider

OpenAI, Anthropic and Google cache automatically — nothing to configure.

LESSON · TRACKING COSTS

How LLM cost is calculated

01

Read token counts

Input, output and subtypes like cache reads or reasoning

02

Identify the model

From ls_model_name and ls_provider in the run's metadata

03

Multiply by price

Per-token prices come from the model pricing table

Most specific token type first

20 input tokens, 5 of them cache reads, plus 10 output tokens. Cache reads are priced first, then the rest of the input at the default rate.

```
# input $2/1M · cache_read $1/1M · output $3/1M
input = 5 × $1/1M + 15 × $2/1M = $0.000035
output = 10 × $3/1M = $0.000030
total = $0.000065
```

The model pricing table

EACH ENTRY · WORKSPACE SETTINGS › MODEL PRICING

Field	What it does
Model name	Human-readable name
Match pattern	Regex matched against ls_model_name
Provider	Matched against ls_provider (optional)
Input / output price	Cost per 1M tokens
Activation date	When the price starts to apply (optional)

Comes pre-filled for most **OpenAI, Anthropic and Gemini** models. You can also:

- Add prices for custom models
- Override with your negotiated rates
- Price cache reads separately

⚠ **Not retroactive:** price changes apply only to traces logged afterwards. No backfill.

Getting token counts in

AUTOMATIC · MOST CASES

Works out of the box

- Calling LLMs with LangChain
- @traceable on OpenAI, Anthropic or OpenAI-compatible calls
- wrap_openai or wrap_anthropic wrappers

No configuration needed.

MANUAL · E.G. COHERE, MISTRAL NATIVE APIS

```
@traceable(run_type="llm", metadata={
    "ls_provider": "my_provider",
    "ls_model_name": "my_model"})
def chat_model(messages):
    result = call_my_model(messages)
    run = get_current_run_tree()
    run.set(usage_metadata={
        "input_tokens": 27, "output_tokens": 13})
    return result
```

Or return usage_metadata in the function's output.

LESSON · TRACKING COSTS

Tracking non-LLM costs

LLM RUNS

Token counts × pricing table

TOOLS, RETRIEVAL, PAID APIS

You pass the literal cost in **total_cost**

```python
@traceable(run_type="tool", name="get_weather")
def get_weather(city: str):
    result = call_weather_api(city)
    run = get_current_run_tree()
    run.set(usage_metadata={"total_cost": 0.0015})
    return result
```

Model calls plus paid APIs — the full cost of every trace.

LESSON · VIEWING COSTS

Three places to see cost

ONE TRACE

Trace tree

Total for the trace, totals per parent run, and a breakdown for every child run

ONE PROJECT

Project stats

Total tokens and cost across all traces in the project

TRENDS OVER TIME

Dashboards

Prebuilt for every project: total cost, split by input and output tokens

Group by any metadata key: customer ID · app version · environment · model

Module 3

Monitoring quality

User sentiment · Errors and latency · A/B testing

LESSON · TRACKING USER SENTIMENT

Fast, error-free, still frustrating

WHAT ERROR AND LATENCY ALERTS WON'T CATCH

- Misunderstood the question
- Correct but unhelpful
- Went in circles for three turns

EXPLICIT · USERS TELL YOU

Thumbs up or down

A button in your app, logged to LangSmith with the SDK

INFERRED · YOU JUDGE IT

Thread-based online eval

An LLM judge reads the conversation and decides if the user seemed satisfied

Logging thumbs up or down

```python
from langsmith import Client
client = Client()

def handle_feedback(trace_id: str, reaction: str | None):
    if not reaction:
        return
    client.create_feedback(
        key="user_sentiment",
        score=1 if reaction == "liked" else 0,
        trace_id=trace_id,
    )
```

trace_id links it

Ties the feedback to that conversation's trace. Surface the ID from your agent's runtime.

Binary is enough

1 or 0, no 1–5 scale. Zero thought for the user means far more responses.

Doesn’t block your app

In Python, trace_id= instead of run_id= sends it in the background.

LESSON · TRACKING USER SENTIMENT

Judge the thread, not the message

	Single-turn evaluator	Thread-based evaluator
Input	One trace	The full conversation thread
Runs when	A new trace arrives	The thread goes idle
Good for	Safety checks, format validation	Topic classification, user satisfaction

JUDGE RULES FROM SESSION 11: One narrow question | Binary or categorical | Aligned with human labels

⚠ **Cost note:** a trace an online eval runs on is upgraded to extended data retention, which affects trace pricing.

The user-sentiment evaluator

+ NEW › NEW EVALUATOR › EVALUATE A MULTI-TURN THREAD

Setting	Value
Name	user-sentiment
Idle time	Same as your other thread evals (project-level)
Sampling	30–50% of threads to start
Model	Gemini 2.5 Flash or GPT-4.1 nano
Messages	Human and AI pairs only
Feedback	user_sentiment · Boolean

THE JUDGE PROMPT, IN SHORT

Positive: thanks, "works now", no lingering questions

Negative: confusion, "still doesn't work", quiet giving up like "I'll figure it out myself"

Neutral ("Okay", "Got it") counts as positive

The final human message weighs most.

Acting on sentiment

EACH BAD CONVERSATION

Automation to an annotation queue

Route negative-sentiment threads to people for review

THE TREND

Scheduled Insights report

Filtered to negative sentiment, to track quality week over week

Thumbs and the judge both score 1 or 0 — so they chart side by side.

Two alerts for two kinds of trouble

THE AGENT FAILS

Error alert

A tool throws, a model API rejects requests, an edge case goes unhandled

→ Linear · a ticket right away

THE AGENT SLOWS DOWN

Latency alert

A tool times out, a model provider degrades, the prompt grows too large

→ Datadog · next to your APM

Users feel a slowdown before you see it — unless an alert tells you.

Error alert to a Linear issue

CREATE THE ALERT

1. Tracing project › Alerts › + New Alert
2. Metric: **Errored Runs**
3. Threshold: **1**, so any error alerts; raise it if transient errors are normal
4. Aggregation window: **5 minutes**
5. Optional: filter to run names or tags

WEBHOOK · BODY TRIMMED

```
POST https://api.linear.app/graphql
Authorization: your-linear-api-key
mutation { issueCreate(input: {
  teamId: "your-team-id",
  title: "LangSmith: Error rate spike detected",
  description: "… {{triggered_metric_value}} …",
  priority: 1 }) { success issue { id url } } }
```

Click **Send Test Alert** and check a test issue appears in Linear.

From ”something broke” to ”here’s why”

IN THE UI

Filter, then open the trace

Filter runs by error status and the alert's time window. The trace tree shows which step errored, with full inputs and outputs.

WITH A CODING AGENT

Query traces in bulk

The LangSmith CLI lets a coding agent pull traces, cluster errors by type and surface patterns you'd miss one trace at a time.

Debugging workflows in depth: session 11, Building Reliable Agents.

Latency alert to a Datadog event

CREATE THE ALERT

1. Tracing project › Alerts › + New Alert
2. Metric: **Latency**
3. Threshold: from your expected response time
4. Aggregation: **Average** over **5 minutes**
5. Optional: only root runs, so child spans don't trigger it

WEBHOOK · BODY TRIMMED

```
POST https://api.datadoghq.com/api/v1/events
DD-API-KEY: your-datadog-api-key
{
  "title": "LangSmith Latency Alert",
  "text": "… {{triggered_metric_value}}ms …",
  "tags": ["source:langsmith", "alert_type:latency"],
  "alert_type": "warning"
}
```

EU: api.datadoghq.eu. Keep the key as a **Workspace Secret**, then **Send Test Alert**.

Why Datadog and LangSmith together?

APM · AROUND THE AGENT

Datadog sees

CPU, memory, network and downstream API health, across your whole stack

When it's slow: is it the infrastructure or a downstream service?

LLM OBSERVABILITY · INSIDE THE AGENT

LangSmith sees

Prompts, completions, tool usage and reasoning steps inside the LLM calls

When it's slow: a bloated prompt, a retry loop, or a degraded provider?

On call, one place: the latency event beside its infrastructure context.

Prebuilt error and latency charts

	Errors	Latency
Overall	Successful vs. errored runs over time	End-to-end trace latency over time
Trend	Trace error rate over time	LLM call latency, to spot provider slowdowns
By tool	Error rate by tool type	Latency by tool call
By run type	Error rate by chain, LLM, tool…	Latency by chain, LLM, tool…

The tool and run-type rows point at the bottleneck — no trace-by-trace digging.

LESSON · A/B TESTING PROMPT VERSIONS

Is the fix actually better?

01

Deploy both

Control and variant, side by side on real traffic

02

Tag every run

prompt_version in the trace metadata

03

Group by version

Monitor tab › metadata.prompt_version

```
version = random.choice(["control", "variant"])
with ls.trace(
    name="support-agent",
    metadata={"prompt_version": version},
) as rt:
    response = run_agent(version, user_question)
    rt.end(outputs={"response": response})
```

Every chart splits

Error rate, latency and feedback scores, both versions side by side

Evals come along free

user-sentiment scores both versions, no extra setup

MODULE 3 · PUTTING IT TOGETHER

A complete quality monitoring system

Lesson 1

User sentiment

Catches the problems metrics miss, through feedback and inferred sentiment

Lesson 2

Error and latency alerts

Catch failures as they happen, routed to Linear and Datadog

Lesson 3

A/B testing

Measures whether a change is really an improvement, on real traffic

Catch failures, understand the experience, and roll out changes on evidence.

THE TAKEAWAY

Know what it costs, whether it works, and whether users like it.

Cost | Latency and errors | Sentiment | A/B tests

SESSION 12

Your plan for today

01

See the cost

Run the sample agent and read cost in its trace tree

02

Price a tool

Add a total_cost to one tool call

03

Capture feedback

Send a thumbs score with create_feedback

04

Run an A/B split

Tag two variants and group the dashboard by them

Course: academy.langchain.com/courses/production-monitoring